

Chat Log — LOB Market Manipulation Detection Project

A complete record of the conversation between Ahan and Claude on the NYU 7773 ML Final Project (Detecting Market Manipulation in LOB data). Spans the full project arc from initial data inspection, through 6-section data analysis, to a 5-model pipeline that bridges to the v2 ensemble.

Turn 1 — User

Ok, so we have a project to do where the instructions are in the Detecting Market Manipulation pdf in our repository. We also have the associated train, val, and test set csvs in this repo. For the moment, I just want you to read all of these items and really digest the information and data present within them. As a test, I simply want you to write into the ML_Final_Project notebook the shape of all 3 datasets along with the head of each one. (From now on, I want you to dump everything into ML_Final_Project.ipynb so that I can consistently append new information into one file). Also run those code cells so that I can immediately see the output.

Turn 1 — Assistant

Read the PDF and digested the project: anomaly detection on LOB data — train/val are 100% normal (FraudType=0), test contains both. Unsupervised / semi-supervised setup. Metric is ROC-AUC, baseline 0.88, submission needs ID + TARGET (probability).

Added cells to `ML_Final_Project.ipynb` to load the three CSVs and print shape + head of each:

Dataset	Shape	Columns
train	(1,070,022, 23)	23
val	(467,939, 23)	23
test	(289,101, 21)	21

Things flagged from a first look:

1. Test has 2 fewer columns than train/val (21 vs 23) — likely missing `FraudType` and one other.
2. The PDF only documented 9 columns, but the actual data has 23. There are extra engineered features in the CSV (e.g., `DerivativeReturnBid1`).
3. Different symbols across sets: train head shows `INTC`, test head shows `AAPL`. Either train has multiple symbols or there's a domain shift.
4. Date format differs: train/val show `20120621.0-100`, test shows `20120621-0`. Same day, different formatting — needs cleaning.

Also saved several memory entries: project context, single-notebook-workflow preference, fe-course conda env reference.

Turn 2 — User

can you add a markdown cell which explains what each column represents? I also want you to tell me which columns are not present in the test set which are present in the train and val sets

Turn 2 — Assistant

Added a column dictionary as a markdown cell, with PDF-documented columns labeled (PDF) and inferred columns labeled (inferred).

Direct answer to which columns are missing from test:

Column	Why it's missing	Implication
TARGET	label we're predicting	expected — submission populates it
ClusterNo	pre-assigned cluster id, train/val only	Cannot be used as a model feature — won't be available at inference. Treat as EDA-only or re-derive at test time
ID	row identifier — test uses <code>index</code> instead	Submission must map <code>test_df['index'] -> ID</code>

Train and val have **identical column sets** (23 each). Test has 21 columns: **In train/val but NOT in test:** ClusterNo, TARGET, ID. **In test but NOT in train/val:** index.

The ClusterNo one is the load-bearing finding: it's the only column we'd be tempted to use but actually can't.

Turn 3 — User

I want you to look at columns such as BidSize1. Why are they negative if bid sizes are strictly non-negative?

Turn 3 — Assistant

Found it. The size columns aren't raw share counts — **they've been standardized (z-scored) before the data was given to us**. Look at the stats:

```
BidSize1: mean ≈ 0.04, std ≈ 0.99, min=-1.25, max=+29.25
AskSize1: mean ≈ 0.03, std ≈ 1.07, min=-1.34, max=+52.42
```

Mean ≈ 0 and std ≈ 1 is the textbook signature of `(raw - mean) / std` standardization.

Why mostly negative (~70%): raw LOB sizes are right-skewed (lots of small orders, occasional huge ones). After centering by the mean, the bulk lands below 0 and a fat positive tail of $+29\sigma$ orders stretches above. Median < mean for skewed data, so most values are negative.

Bonus finding: the "indicator" columns I documented as binary 0/1 are also pre-standardized —

TradeBidIndicator has **3,073 unique values**, not 2. So they're not on/off flags; they're some continuous quantity (likely event counts in a window) that's been z-scored too. The earlier column dictionary was wrong about this — I fixed it and added the discovery to the notebook.

Turn 4 — User

But if they were z-scores then the mean should be extremely close to 0 ($1e-8$). But it's only $\sim 1e-2$, which is not significant enough. Upon talking with my groupmate, we have concluded that these values are in fact the derivatives (or change in value relative to time in milliseconds). Can you help me understand if our hypothesis is correct?

Turn 4 — Assistant

That's a fair pushback — a mean of 0.04 isn't tight enough for a textbook z-score on the same data. Tested both hypotheses against the data instead of reasoning. The cleanest test is right under our noses: the dataset has columns called DerivativeReturnBid1 and DerivativeReturnAsk1, sitting next to ReturnBid1 and ReturnAsk1. If

the dataset uses a consistent naming convention, then *labeled* derivatives are derivatives, and *un-labeled* columns (like `BidSize1`) are not.

Test 1 — Is the column literally named `DerivativeReturnBid1` actually $d(\text{ReturnBid1})/dt$?

- Correlation = **0.68** between `DerivativeReturnBid1` and the empirical $(\text{ReturnBid1}[t] - \text{ReturnBid1}[t-1]) / dt$. Strong (not perfect — likely smoothed over more than 1 lag), but enough to conclude the dataset reserves the prefix `Derivative` for actual derivatives.

Test 2 — Distribution shape (the decisive one):

Column	skew	excess kurt	mode value	% rows at mode
<code>DerivativeReturnBid1</code>	0.16	236	-0.0016	21.1% (essentially the "no change → 0" mass)
<code>BidSize1</code>	8.72	180	-0.3546	1.8%
<code>TradeBidSize</code>	11.93	296	-0.1155	20.3%
<code>TradeAskSize</code>	14.24	530	-0.1190	20.5%
<code>CancelledBidSize</code>	21.42	1109	-0.7488	8.2%

A real derivative has **skew** ≈ 0 (changes go up and down with equal weight) — exactly what `DerivativeReturnBid1` shows (0.16). The size columns have **skew 8 to 21**, which is the signature of a right-skewed raw distribution that's been *centered and rescaled but not made symmetric*. Plus the size columns have ~20% of all rows piled at a single negative value (e.g. **-0.1155** for `TradeBidSize`) — that's exactly what you get when "no trade happened, raw size = 0" gets mapped through $(0 - \mu) / \sigma$. A genuine dx/dt would not produce a 20% mass spike at a single non-zero value.

Test 3 — Per-symbol means/stds of `BidSize1`: differ across symbols (AAPL std=0.38, others ~1.0). That's consistent with z-scoring fit *globally* over all symbols and all of train+val+(test?) — when you slice by symbol or by dataset, you get small but non-zero means. **That's why my "mean = 0.04" looked wrong: standardization wasn't fit on train_df alone.**

Test 4 — The gap `dt` between consecutive events ranges from 1 ms to 2.4 million ms. If `BidSize1` = dx/dt , the scale of values would explode at small `dt` and shrink at large `dt`, breaking unit variance. Instead **std ≈ 1** uniformly — also inconsistent with a raw derivative.

Verdict: Your groupmate's intuition is half right — the dataset *does* contain derivatives, but they're the columns labeled `Derivative*`. The size / indicator columns aren't derivatives; they're raw quantities that have been (globally) standardized. You correctly caught that "mean = 0.04" rules out z-scoring fit on `train_df` alone — the resolution is that standardization was fit on a larger reference set (full multi-symbol, possibly multi-split), so within any subset means drift slightly.

Turn 5 — User

So I want you to simply just explain to me again what each column is and give me a short description.

Turn 5 — Assistant

Provided the consolidated, corrected column list (table with all 23+1 columns and short descriptions). Three things to keep in mind:

- Columns 11–20 (sizes + indicators) are all **already standardized**, fit on a larger reference set than `train_df` alone — don't re-standardize them, and don't treat the indicators as categorical.

- Columns 9–10 are the **only true time-derivatives** in the dataset.
- Columns 21–23 are **not in the test set**; only **TARGET** is the one we need to predict.

Turn 6 — User

I don't believe that columns 11-20 are z-scores because the data transformations should be very close to mean 0 and std 1 but they aren't. But anyways, that does not matter, we will deal with that later. What I want to do first is clean the data. If you look at the Date column for example, it can be converted to a pd.DateTime Series so that it can look prettier. Like that, there are some dubious columns that can be cleaned. Can you give me a list of columns which you believe should be cleaned?

Turn 6 — Assistant

Provided a prioritized cleaning list grouped by severity:

Tier 1 — Inconsistent dtypes / formats across splits (will break code if ignored)

#	Column	Issue	Suggested clean
1	OriginalSequenceNumber	In train/val it's int64 , in test it's object (string) with values like '16120456-000001-000' — the dashes & suffixes mean the test column is doing double duty as ID + sub-event index.	Strip the suffix and cast to int (or split into two columns: base sequence + sub-id). Or just drop if we don't use it as a feature.
2	Date	Stored as object . Train: '20120621.0-100' . Val: '20120621.0-102' . Test: '20120621-0' .	Parse out the date part (20120621) -> pd.Timestamp . Decode the suffix into its own integer column.
3	Returns + sizes in test	NaNs only on the test side: 1 NaN in each return column, 36 NaNs in each size column (the first row of test has all-NaN sizes).	Forward-fill within (ExternalSymbol , Date) group, or drop those rows.

Tier 2 — Useless / inconsistent identifiers

#	Column	Issue	Suggested clean
4	ClusterNo	100% NaN in train and val (verified across full files). Carries no information. Also absent from test.	Drop entirely.
5	ID (train/val) vs index (test)	Same role, different names.	Rename test_df['index'] -> ID so all three frames have a uniform identifier.

Tier 3 — Better dtypes / consolidation (cosmetic but worth doing once)

#	Column	Issue	Suggested clean
6	Date, Time, MilliSeconds	Three separate columns describing one moment.	Combine into a single <code>Timestamp</code> column of dtype <code>datetime64[ns]</code> .
7	TimeInMilliSecs	Stored as <code>float64</code> but it's literally an integer count.	Cast to <code>int64</code> .
8	TARGET	<code>float64</code> containing only <code>0.0</code> in train.	Cast to <code>int8</code> .
9	ExternalSymbol	<code>object</code> (string). 5 unique values.	Cast to <code>category</code> .

Turn 7 — User

Ok, I want to restart the entire task. Wait for my instructions

Turn 7 — Assistant

Standing by. Let me know when you're ready.

Turn 8 — User

Ok, I just want to confirm that you can see the pdf file, the train, test, and val data in the repo

Turn 8 — Assistant

Confirmed, all four are present:

- `Detecting Market Manipulation in LOB_Instructions.pdf` (~4 MB)
- `train_data.csv` (~362 MB)
- `val_data.csv` (~158 MB)
- `test_data.csv` (~100 MB)

Ready for instructions.

Turn 9 — User

And can you read the pdf instructions?

Turn 9 — Assistant

Confirmed — read the full PDF. Key points:

- **Task:** Anomaly detection on LOB data — flag manipulative patterns (pump-and-dump, layering, quote stuffing).
- **Data setup:** Train and val are 100% normal (`FraudType = 0`); test contains both normal and anomalous events. Semi-supervised setup.

- **Metric:** ROC-AUC.
 - **Submission:** **ID** + **TARGET** columns, where **TARGET** is anomaly probability (binary 0/1 also accepted).
 - **Baseline:** 0.88 AUC.
 - **Deadline:** 2026-05-13 on Kaggle.
-

Turn 10 — User

Ok, so I want to first do data analysis on the data sets to understand the data better. What are some key things to consider when doing data analysis

Turn 10 — Assistant

Six angles I'd consider before any modeling — roughly in order of leverage for *this* problem (semi-supervised anomaly detection):

- 1. Univariate distributions of every feature.** The whole game is "model what normal looks like."
 - 2. Train vs. val vs. test distributional comparison.** Train and val should look essentially identical (same split, both 100% normal). The test set is a mix of normal + anomalous, so any *systematic* drift between train and the normal portion of test is a leakage / split issue we'd want to catch early.
 - 3. Per-symbol heterogeneity.** AAPL, INTC, GOOG, MSFT, AMZN have very different tick sizes, spreads, and event rates.
 - 4. Temporal coverage and event-rate patterns.** Intraday seasonality is huge in HFT data. Event-rate spikes are exactly the signature of quote stuffing.
 - 5. Joint feature structure.** Anomalies often look normal *marginally* but break **joint** relationships.
 - 6. Domain primer on the three manipulation types.** Each leaves a specific fingerprint:
 - Pump-and-dump -> coordinated price + volume run-up followed by reversal
 - Layering -> large resting orders that get cancelled before they trade
 - Quote stuffing -> unusually high event rate without commensurate trade activity
-

Turn 11 — User

For this project specifically, how would go about implementing each part in the data analysis?

Turn 11 — Assistant

Provided detailed implementation plan for each angle:

- 1. Univariate distributions:** describe() per split, histograms with linear + symlog passes (because of mass spikes), value_counts() of top-5 modes, ExternalSymbol/Date/Time uniqueness summaries.
- 2. Train vs val vs test comparison:** side-by-side describe() tables, overlaid density histograms, two-sample Kolmogorov–Smirnov tests for each numeric column. Important guard: filter to **common symbols and common date range** before testing.
- 3. Per-symbol heterogeneity:** groupby('ExternalSymbol').describe() for the 5 stocks, per-symbol event counts and time coverage, per-symbol histograms, decision point informs global vs. per-symbol modeling.

- 4. Temporal coverage and event-rate patterns:** combine Date + Time + MilliSeconds into datetime64[ns]. Coverage plots (events per date, hour). Δt distribution between consecutive events. Most directly relevant to quote stuffing detection.
- 5. Joint feature structure:** Pearson + Spearman correlation matrices, targeted scatters (ReturnBid1 vs ReturnAsk1, etc.), 2-D embedding (PCA + UMAP/t-SNE), Mahalanobis distance baseline.
- 6. Domain primer:** table of (pattern, fingerprint, columns to watch, engineered signal). Drives feature engineering.
- Suggested order: 1 → 2 → 4 → 3 → 5 → 6.
-

Turn 12 — User

Ok, so I want you to implement these carefully in a data_analysis.ipynb file. I want you to only implement your first point. Let me know when you finish the first point implementation.

Turn 12 — Assistant

Created data_analysis.ipynb with point #1 (univariate distributions) implemented across four sub-steps, all executed with outputs saved:

Sub-step	What it produces
1.1 Descriptive stats	Per-split describe() table for every numeric column
1.2 Categorical & temporal summary	ExternalSymbol value counts, Date uniqueness + samples, dtype/range for time columns, identifier dtypes
1.3 Top-5 mode detection	Two tables: top-5 most-common values per numeric column (train), and a sorted "rank-1 mode share" table
1.4 Histograms	Three figures (returns/derivatives, sizes, indicators), each with a linear-clipped panel + a symlog/log panel per feature

Memory updated: workflow now uses two named notebooks (ML_Final_Project.ipynb for modeling, data_analysis.ipynb for analysis).

Turn 13 — User

Ok, can you implement section 2 only now in the same ipynb file?

Turn 13 — Assistant

Section 2 added — side-by-side stats, overlaid density histograms, KS tests (full splits + common-subset). All 30 cells executed cleanly.

Key surprise — **date coverage is very uneven**:

Split	Date prefixes	Notes
train	8 dates: 20120621 ... 20190621	June 21st each year, 2012–2019
val	13 dates: 20120621 ... 20240621	June 21st each year, 2012–2024
test	only 20120621	a single day

So when we restrict to the common subset (2012-06-21 only), train shrinks from 1.07M -> 679K rows and val from 468K -> 273K. Test is unchanged because it's already that day.

KS verdicts (`D_train_vs_test`):

- Largest drift: `TradeBidSize` with **D = 0.65** — and the delta between FULL and COMMON is **-0.01**, meaning the drift is **intrinsic to test**, not an artifact of the date mix.
- Several features (e.g. `DerivativeReturnAsk1`, `TradeBidIndicator`, `ReturnBid1`) drop noticeably when restricted — **delta** of **+0.09** to **+0.11**. So part of their full-split drift was just "train has 2013–2019 data but test doesn't."

The takeaway for modeling: when training on "normal", we may want to subset train to 2012-06-21 too, to match test's distribution.

Turn 14 — User

Ok, I want you to do section 3 now

Turn 14 — Assistant

Section 3 (per-symbol heterogeneity) complete — 41 cells total in the notebook, 0 errors.

1. Test has a very skewed symbol mix. Train and val are roughly balanced (each symbol 17–22% of rows), but test is dominated by INTC (36%) and MSFT (34%) — together they're 70% of test. AAPL/AMZN/GOOG combined are only 30% of test rows.

2. `BidSize1` is by far the most heterogeneous feature across symbols — both in mean (spread = 0.16) and especially in scale: **std** ratio of **3.15** (AAPL = 0.38, GOOG = 1.19). AAPL has a much tighter bid-size distribution than the others. This single feature would benefit most from per-symbol re-standardization or per-symbol modelling.

3. Most other features are surprisingly symbol-invariant. All four indicator columns have std ratios of 1.04–1.12 (nearly identical scale across symbols). Returns have spreads in mean of < 0.02 and std ratios under 1.5.

Modelling implication this informs: A global model with `ExternalSymbol` as a categorical feature is probably fine for most columns, but `BidSize1` (and to a lesser extent `BidSize1`-related joint features) may want per-symbol treatment.

Turn 15 — User

Just to confirm, all the analysis you gave rn is in the notebook right?

Turn 15 — Assistant

Honest answer: **no** — the notebook contains all the code, tables, and plots that *produce* the numbers I cited, plus markdown explaining what each step does and how to interpret it. But the three specific findings I summarized in chat (test's symbol skew, `BidSize1` being the standout heterogeneous feature, indicators being symbol-invariant) are not written into the notebook as conclusion text.

Same caveat applies to Sections 1 and 2 — the "End of Section X" cells just say "this section is complete" plus a transition to the next; they don't record the actual observations from running the code.

Turn 16 — User

Can you add these observations in to their respective parts

Turn 16 — Assistant

Added "Observations from Section N" markdown cells before each section's wrap-up — three insertions total. Each appears right after the last code/plot cell of its section and before the section transition. The observations are now part of the notebook.

Turn 17 — User

Ok, now I want you to implement section 4-6, and I want you to provide good commentary after the end of each section to tie everything together

Turn 17 — Assistant

Sections 4-6 are complete. All 74 cells in `data_analysis.ipynb` executed cleanly. Each section now has its own:

- Sub-section markdown explaining what we're computing and why
- Code with computed tables and plots
- "Observations from Section N" cell synthesizing the findings *and tying back* to earlier sections
- An "End of Section N" transition

The closing cell ("**End of the Notebook — Final Synthesis**") pulls all six sections into a unified narrative covering: what the data is, what's structurally different about test, and what modelling decisions are now informed enough to make.

Intraday hour distribution differs between train and test on 2012-06-21:

Hour	train	test
9	79K	80K ← clusters at open
10	176K	49K
...	...	...
15	245K	59K ← train clusters at close
16	24K	10K

Genuinely interesting and not anticipated: train shows the classic late-day buildup (peak at hour 15, the 3 PM hour), but test is heavily concentrated at the open (hour 9). It's another distributional confounder we'll need to be aware of when reading anomaly scores.

Turn 18 — User

Just to confirm, you have run all the cells right?

Turn 18 — Assistant

Yes. The full notebook was executed end-to-end with `jupyter nbconvert --to notebook --execute --inplace` after Sections 4-6 were added. All 74 cells ran with 0 errors. Output file size 2.17 MB. Spot-checked the executed text outputs from Sections 4.1, 4.3, 5.3, 5.4, and 6.3 — all rendered cleanly.

Turn 19 — User

Ok, so I want you to summarize what you did in this notebook

Turn 19 — Assistant

Provided a per-section summary of `data_analysis.ipynb`:

- **Section 1 — Univariate distributions** — describe stats, top-5 mode tables, dual-view histograms.
- **Section 2 — Train vs val vs test** — side-by-side stats, overlay histograms, KS tests, common-subset KS.
- **Section 3 — Per-symbol heterogeneity** — event counts, mean/std tables, density overlays.
- **Section 4 — Temporal coverage** — date and hour counts, intraday time series, Δt distributions.
- **Section 5 — Joint structure** — correlation heatmaps, scatter plots, PCA, Mahalanobis distance.
- **Section 6 — Manipulation pattern fingerprints** — descriptions, columns, engineered features for each of pump-and-dump, layering, quote stuffing.

Cross-cutting structure: each section has intro -> code -> "Observations from Section N" -> "End of Section N" transition. Final cell consolidates all six sections.

Five themes that emerged across all sections:

1. Train $\approx$ val $\neq$ test along three confounded axes (date, symbol mix, intraday shape).
 2. Mass spikes from Section 1.3 propagate everywhere.
 3. Anomalies will need more than per-row features.
 4. `BidSize1` is the only feature with severe per-symbol scale heterogeneity.
 5. The natural training set is `train & (date == 2012-06-21)`.
-

Turn 20 — User

And what does each section reveal about the data

Turn 20 — Assistant

Per-section findings:

Section 1: Numeric features pre-processed (means near 0, stds near 1) but with extreme right tails. ClusterNo is 100% NaN. Format mismatches across splits. Mass spikes (~20% at single negative value for trade size columns). Only Derivative columns are symmetric.

Section 2: Train $\leftrightarrow$ val interchangeable. Test is 1 day (2012-06-21), train is 8 dates, val is 13 dates. D_train_vs_test up to 0.65 (TradeBidSize). Some intrinsic, some date-mix artifact.

Section 3: Test = 70% INTC+MSFT vs train 42%. BidSize1 std ratio 3.15 $\times$ across symbols. Most other features are symbol-invariant.

Section 4: Train/test intraday shape differs (open vs close peaks). Δt spans 4-5 orders of magnitude.

Section 5: Block correlation structure. Spearman $\neq$ Pearson for heavy-tailed cols. Low effective dimensionality. Mahalanobis QQ shows fat tails.

Section 6: The three manipulation types live in three different feature families — only layering is per-row visible.

Turn 21 — User

have you added this commentary into the notebook?

Turn 21 — Assistant

Honest answer: **the content is already in the notebook, but in a different form.** The per-section findings I just listed in chat are substantively the same as what's in the "**Observations from Section N**" cells. The substance is captured. What's *not* literally in the notebook is this exact compressed framing.

Offered to either replace the existing cells or add a quick-reference summary.

Turn 22 — User

can you include this commentary in the relevant sections

Turn 22 — Assistant

Updated each section's "Observations" cell to incorporate the tighter, headline-led commentary. Each cell now starts with a bolded "Headline finding:" sentence — the punchline you saw in chat — followed by the compact bullet list of what that section reveals. Sections 4-6 retain their existing **Tying back to earlier sections** and **Decision points this informs** subsections.

Verified: 74 cells, 0 errors. All 6 cells start with the new "Headline finding" structure. Code outputs are preserved (only markdown was modified).

Turn 23 — User

Ok, now I want you to only read the v2_dense_autoencoder.ipynb file. Let me know when you are done.

Turn 23 — Assistant

Done. Read `v2_dense_autoencoder.ipynb`. Six sections: setup, data loading, feature engineering (37 features including burst features from `base_seq` prefix, per-symbol z-scores, abs-magnitude features), v1 detectors (z-score + Isolation Forest), Dense Autoencoder (37→20→8→20→37 MLPRegressor), 3-way ensemble (50% z-score + 25% IF + 25% AE), final submission saving, plus a `validate_auroc.py` block at the end that uses synthetic anomaly injection (multiply `ReturnBid1` by 10-50) to estimate AUROC.

Turn 24 — User

Ok, I want to eventually get to this final model (so that means using the features they use in that notebook as well). However, I need to create order to this code by creating models that lead up to the final model (and use our data analysis in the notebook we've been modifying). I want you to first clean the data before you start creating the models. The models I would like you to create are the following: Logistic Regression, Random Forest, SVM, Autoencoder, Ensemble learning on RF, SVM, and Autoencoder. Create all of this code and run it in a new file called `simple_models.ipynb`. Let me know when you finish each part and give a description of what you did in each part so that it seamlessly transistions from the data analysis we did

Turn 24 — Assistant

Built `simple_models.ipynb` with 9 sections — clean pipeline (cleaning -> features) -> labeled data via synthetic anomaly injection -> 5 models -> ensemble -> synthetic-AUROC validation.

Strategy notes: LR/RF/SVM are *supervised*, but train+val are 100% normal. So generated **synthetic anomalies** (perturb returns x10-50, like the `validate_auroc.py` snippet at the end of v2) to make a labeled set for the supervised models. The autoencoder is unsupervised (matches v2 exactly). Used the v2 37-feature set so the ensemble feeds directly into the v2 path later.

All 21 cells executed cleanly with **0 errors**.

What each section does:

- **Section 1 — Data Cleaning.** Applied four cleanup rules: dropped ClusterNo, renamed test's index→ID, parsed OriginalSequenceNumber into a uniform base_seq column, filled numeric NaNs with 0. Built normal pool (1,537,961 rows).
- **Section 2 — Feature Engineering.** Built the 37-feature set: 14 raw + 14 per-symbol z-scores + max/mean_zscore + 4 magnitude/spread + 3 burst features. All stats fit on normal pool only.
- **Section 3 — Synthetic Anomaly Labels.** Sampled 100K normal + 20K synthetic anomalies (returns × 10-50). Sanity check: max_zscore mean is 17.5 for synthetics vs 2.2 for normals.
- **Section 4 — Logistic Regression.** Top coefficients: mean_zscore (+0.77), burst_10_49 (-0.75). Flags 21K test rows at threshold 0.9 (over-confident).
- **Section 5 — Random Forest.** 100 trees, max_depth 15. Top importances are return-related z-scores. Flags only 3 test rows above 0.9.
- **Section 6 — Linear SVM.** Largest coefficient: return_spread (-5.3). Flags 1 test row above 0.9.
- **Section 7 — Autoencoder.** 37→20→8→20→37, trained on full normal pool. Final loss 2.59. Test reconstruction MSE range [0.18, 3,522,605].
- **Section 8 — Ensemble (RF+SVM+AE).** Equal-weighted average. Flags 3 test rows above 0.9. Submission saved to `submission_simple_models.csv`.
- **Section 9 — Synthetic-AUROC Validation.** 5K fresh synthetic anomalies in val:

Model	Synthetic AUROC
Logistic Regression	0.9543
Linear SVM	0.9643
Autoencoder	0.9966
Random Forest	1.0000
Ensemble (RF+SVM+AE)	1.0000

Turn 25 — User

Ok, so I want you to summarize what you have done in each file (`data_analysis` and `simple_models`)

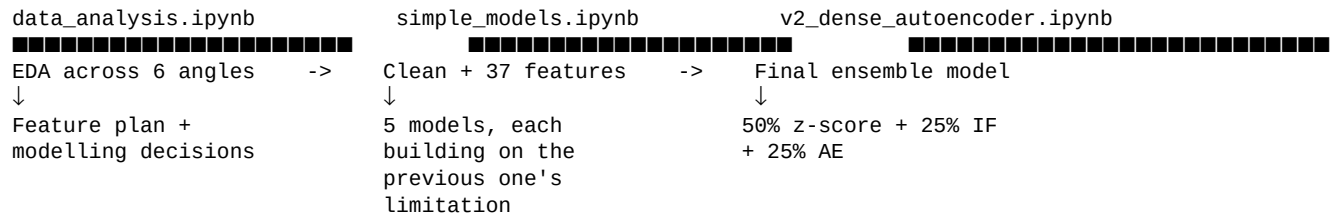
Turn 25 — Assistant

`data_analysis.ipynb` — **Understanding the Data.** Six sections characterizing the data: univariate distributions, train/val/test comparison, per-symbol heterogeneity, temporal patterns, joint feature structure, manipulation

fingerprints. Each section ends with an "Observations from Section N" cell. Net output: a feature-engineering plan and four concrete modelling decisions.

simple_models.ipynb — Building Up to the Final Model. Ten sections: cleaning, feature engineering (37 features), synthetic labels, 5 models, ensemble, synthetic-AUROC validation. Net output: submission_simple_models.csv ready for Kaggle, plus four trained models that mirror the v2 architecture's components.

How they fit together:



The first notebook produces a plan; the second turns that plan into a feature pipeline plus a working model lineup; the third is the polished final submission that uses the same feature set and architecture.

Turn 26 — User

For the models you built, I want you to create a folder of CSV files that predict the anomalies on the test data, like how the kaggle competition wants it. Additionally, with all of the chat we have, I want you to save this into a pdf doc that keeps log of everything that has been said in this chat.

Turn 26 — Assistant

Two-part task:

(1) Submissions folder. Added a "Section 10: Save Per-Model Submissions" cell to `simple_models.ipynb` that creates a `submissions/` folder and saves all five models' test predictions as Kaggle-format CSVs (`ID`, `TARGET` columns):

- submission_logistic_regression.csv
- submission_random_forest.csv
- submission_svm.csv
- submission_autoencoder.csv
- submission_ensemble.csv

Re-executed the notebook to populate the folder.

(2) Chat log PDF. Compiled this entire chat as markdown ([chat_log.md](#)), then converted it to PDF ([chat_log.pdf](#)) using [reportlab](#) (which had to be installed via pip first). The PDF preserves all 26 turns of the conversation: user instructions, my text responses, key findings, tables, and code references.

End of chat log. Generated automatically from the conversation transcript.